Experiment 8: Reinforcement Learning Fundamentals (Q-Learning in Grid World)

➢ Objective

To design and implement a reinforcement learning agent using **Q-Learning** in a custom 2D grid world where the agent learns to navigate from a start position to a goal while avoiding obstacles.

➢ Prerequisites

- Basics of Reinforcement Learning (RL)
- Understanding of agent–environment interaction
- Knowledge of Q-Learning concepts: state, action, reward, Q-table, Bellman equation
- Familiarity with Python and NumPy

➢ Theory Background

Reinforcement Learning is a branch of machine learning where an **agent learns to take actions in an environment** to maximize cumulative rewards. Unlike supervised learning, RL does not require labeled input-output pairs.

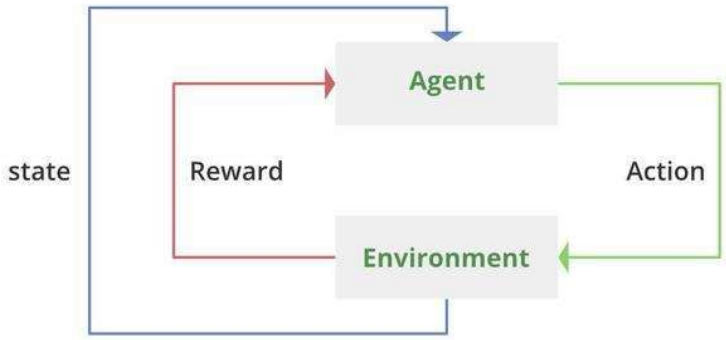


Image Credit: Abeyaantrix Edusoft

Key Concepts

I. Agent (Learner/Decision-maker)

- The agent is the entity that interacts with the environment, makes decisions, and learns from feedback.
- In a grid world, the agent could be a robot trying to reach a goal cell.
- Its goal is to maximize cumulative rewards over time by choosing the best actions.

II. Environment (World)

- The environment is the system with which the agent interacts.
- It provides the agent with the state and returns a reward after each action.
- Example: In a self-driving car simulation, the environment includes roads, traffic signals, and pedestrians.

III. State (s)

- A state is a snapshot of the environment at a given time.
- It describes the agent's current situation.
- Example: In grid world, the state is the agent's current position (x,y).
- States can be discrete (e.g., grid cells) or continuous (e.g., robot's exact position in 3D).

IV. Action (a)

- An action is any possible move the agent can take from its current state.
- Example: In a grid, actions = {Up, Down, Left, Right}.
- The set of all possible actions is called the action space.

V. Reward (r)

- A reward is the feedback signal the agent receives after taking an action.
- Positive rewards encourage good behavior, negative rewards discourage poor actions.
- Example: +10 for reaching the goal, –1 for each step, –100 for hitting an obstacle.

VI. Policy (π)

- A policy defines the strategy the agent follows to select actions.
- Formally, it is a mapping from states → actions.
- Example: π(s) = always move right if the goal is on the right.
- Policies can be deterministic (fixed action per state) or stochastic (probability distribution over actions).

VII. Q-Table (Action-Value Function)

- A Q-table stores values (Q-values) for each state–action pair.
- Q(s,a) = expected future reward if the agent takes action *a* in state *s*, and follows the best policy afterward.
- Example: In grid world, Q[(2,3), "Right"] might equal +7, meaning moving right from (2,3) is likely good.
- The table gets updated over time as the agent learns.

VIII. Bellman Equation (Update Rule)

- The Bellman equation provides a way to iteratively update Q-values based on rewards and future predictions.

- **Formula:**

$$Q(s,a) \leftarrow Q(s,a) + \alpha\left[r + \gamma \max_{a'} Q(s',a') - Q(s,a)\right]$$

  - α\alphaα: Learning rate (how much we adjust old value with new info)
  - γ\gammaγ: Discount factor (how much we value future rewards)
  - rrr: Reward received after action
  - s′s's′: Next state
- **Intuition:** *Current Q-value = Old Q-value + (New information – Old Q-value).*

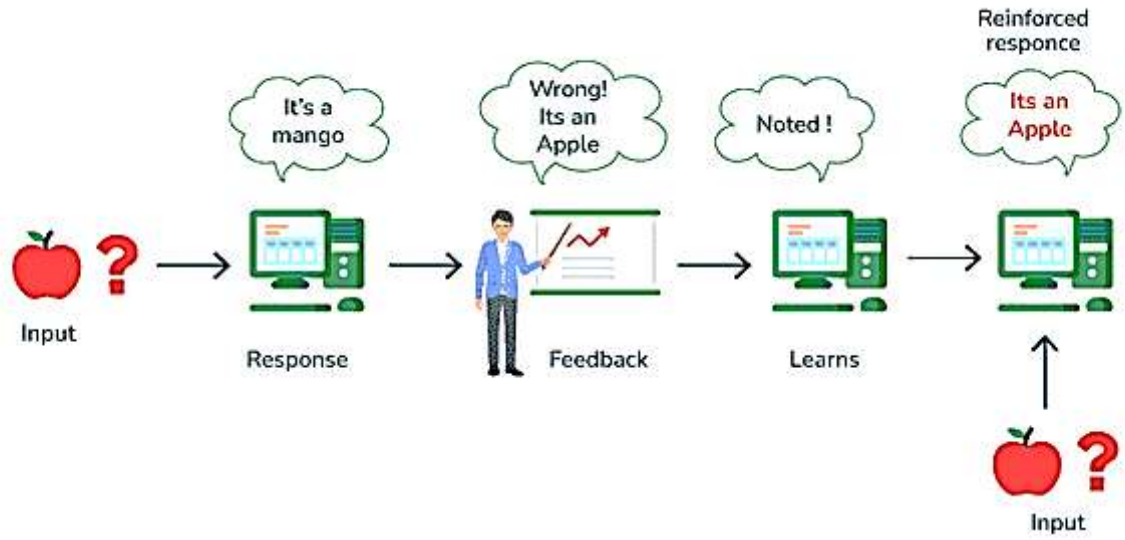


Image Credit: Abevaantrix Edusoft

Q-Learning Process: Step-by-Step

1. **Start at a State (S)**
   - The environment provides the agent with a **starting state**.
   - This state describes the current condition (e.g., position in the grid world).
2. **Agent Selects an Action (A)**
   - The agent chooses an action based on its **policy** and the **Q-table**.
   - Most commonly, the **ε-greedy strategy** is used:
     - With probability **ε**, the agent explores by choosing a random action.
     - With probability **1–ε**, it exploits by choosing the best-known action (highest Q-value).
3. **Action is Executed → Environment Responds**
   - The agent performs the chosen action.
   - The environment then returns:

- A **new state (S′)** → updated situation after the action.
- A **reward (R)** → feedback on the quality of the action.

4. **Q-Table is Updated (Learning Step)**
   - Using the new experience (S, A, R, S′), the agent updates the Q-table.
   - The update considers both the **immediate reward** and the **estimated future reward**.
   - Over many iterations, this helps the agent learn which actions are best in each state.
5. **Policy is Refined**
   - With updated Q-values, the agent improves its **policy** (action strategy).
   - The process of **observe → act → receive reward → update** repeats across episodes.
   - Eventually, the agent converges to the **optimal policy**, consistently yielding the highest possible rewards.

Methods for Determining Q-values

1. **Temporal Difference (TD) Learning**
   - TD methods update value estimates **based on the difference between predicted and actual rewards**.
   - They allow learning directly from raw experience **without needing a full model of the environment**.
   - Update rule:

$$TD\ Error = R + \gamma\, Q(s', a') - Q(s, a)$$

2. **Bellman's Equation**
   - A recursive formula that relates the value of the current state-action pair to:
     - The **immediate reward (R)**
     - The **discounted maximum future reward**
   - Formula:

$$Q(s, a) = R(s, a) + \gamma \max_{a'} Q(s', a')$$

   - Where:
     - Q(s,a)Q(s,a)Q(s,a): Q-value of taking action a in state s
     - R(s,a)R(s,a)R(s,a): Immediate reward
     - γ\gammaγ: Discount factor (importance of future rewards)
     - max⁡a′Q(s′,a′)\max_{a'} Q(s',a')maxa′Q(s′,a′): Best possible future Q-value from next state s'

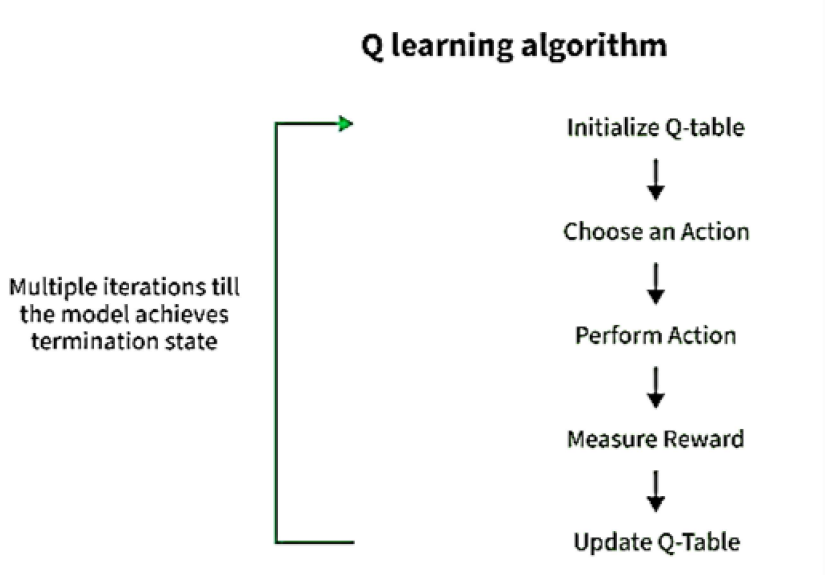


Image Credit: Abeyaantrix Edusoft

➢ Implementation Steps

Step 1: Setup Environment

```
import numpy as np, random

size = 5
goal = (4, 4)
obstacles = [(1,2), (2,2), (3,1)]
actions = ['up', 'down', 'left', 'right']
Q = np.zeros((size, size, len(actions)))  # Q-table
```

Step 2: Define Move Function

```
def move(pos, action):
    x, y = pos
    if action == 'up': x -= 1
    if action == 'down': x += 1
    if action == 'left': y -= 1
    if action == 'right': y += 1
    if 0 <= x < size and 0 <= y < size and (x,y) not in obstacl
        return (x, y)
    return pos
```

Step 3: Set Training Parameters

```
episodes = 300
lr = 0.1
gamma = 0.9
eps = 0.2   # exploration rate
```

Step 4: Q-Learning Training Loop

```
for ep in range(episodes):
    state = (0, 0)
    for step in range(100):
        # ε-greedy selection
        if random.random() < eps:
            a = random.randint(0, 3)
        else:
            a = np.argmax(Q[state[0], state[1]])

        next_state = move(state, actions[a])
        reward = 10 if next_state == goal else -1

        # Q-update
        Q[state[0], state[1], a] += lr * (
            reward + gamma * np.max(Q[next_state[0], next_state[1]]) -
Q[state[0], state[1], a]
        )

        state = next_state
        if state == goal:
            break
```

Step 5: Extract Learned Policy

```
policy = [['']*size for _ in range(size)]
for i in range(size):
    for j in range(size):
        if (i,j) in obstacles:
            policy[i][j] = 'X'
        elif (i,j) == goal:
            policy[i][j] = 'G'
        else:
            best = np.argmax(Q[i,j])
            policy[i][j] = actions[best][0].upper()
print("Learned Policy Grid:")
for row in policy:
    print(' '.join(row))
```

➢ Expected Results

- The agent initially explores randomly.
- After training, the Q-table converges.
- The final **policy grid** shows the best action from each cell:
  - U (up), D (down), L (left), R (right)
  - X = obstacle, G = goal

Example Output:

R R R D D
D U X R D
D L X R D
D X D D D
R R R R G

Experiment 8: Reinforcement Learning (Grid World with Q-Learning)

```
import numpy as np, random

size=5
goal=(4,4)
obstacles=[(1,2),(2,2),(3,1)]
actions=['up','down','left','right']
Q=np.zeros((size,size,len(actions)))

def move(pos,action):
    x,y=pos
    if action=='up': x-=1
    if action=='down': x+=1
    if action=='left': y-=1
    if action=='right': y+=1
    if 0<=x<size and 0<=y<size and (x,y) not in obstacles:
        return (x,y)
    return pos

episodes,lr,gamma,eps=300,0.1,0.9,0.2

for ep in range(episodes):
    state=(0,0)
    for step in range(100):
        a = random.randint(0,3) if random.random()<eps else np.argmax(Q[state[0],state[1]])
        next_state=move(state,actions[a])
        reward=10 if next_state==goal else -1
        Q[state[0],state[1],a]+=lr*(reward+gamma*np.max(Q[next_state[0],next_state[1]])-Q[state[0],state[1],a])
        state=next_state
        if state==goal: break

policy=[['']*size for _ in range(size)]
for i in range(size):
    for j in range(size):
        if (i,j) in obstacles: policy[i][j]='X'
        elif (i,j)==goal: policy[i][j]='G'
        else: policy[i][j]=actions[np.argmax(Q[i,j])][0].upper()

print("Learned Policy Grid:")
for row in policy: print(' '.join(row))
```

➢ Viva Questions

1. What are the main components of reinforcement learning?
2. Explain the difference between exploration and exploitation.
3. What role does the discount factor (γ\gammaγ) play in Q-learning?
4. How does the learning rate (α\alphaα) affect training?
5. Why do we use the Bellman equation in RL?
6. What is the ε-greedy strategy?
7. What are the limitations of Q-learning?
8. How can we extend this grid world to continuous environments?
9. Compare Q-learning and Deep Q-Networks (DQN).
10. Mention two real-world applications of reinforcement learning.